

Automated Vendor SLA Tracker

Group 38: Ryan Pham

Course: CS4365 Intro to Enterprise Computing

Term: Spring 2026

Table of Contents

Introduction.....	3
Motivation.....	3
Proposed Concept.....	4
Related Work.....	4
System Architecture.....	5
Enterprise Configuration & Scalability:.....	7
Implementation Methodology.....	7
Data Extraction & Transformation Pipeline:.....	7
Database Configuration & Business Intelligence Integration:.....	8
Alerting Microservice:.....	9
Evaluation and Results.....	10
System Load Testing:.....	10
Alert Throttling Validation:.....	10
Future Work.....	11
Project Deliverables.....	12
Skill Learning.....	13
References.....	14

Introduction

Modern enterprise operations heavily rely on third-party vendors for logistics, shipping, and supply chain fulfillment. To govern these relationships, organizations establish Service Level Agreements (SLAs) which legally bind vendors to specific delivery timeframes and performance thresholds. Tracking these metrics is critical to ensuring business efficiency, facilitating dynamic operational planning, and maintaining strict cost control. The financial stakes for failing to actively monitor these SLAs are massive. According to McKinsey & Company, major supply chain disruptions occur on average every 3.7 years and can compromise a company's profits by as much as 40% over a decade [1]. Despite these risks, a 2025 McKinsey survey revealed that while 95% of supply chain leaders have visibility into their direct Tier 1 vendors, only 42% have visibility into deeper layers of their supplier network [2].

Motivation

Traditionally, companies have relied on reactive, passive methods to monitor these critical vendor compliance metrics. Standard business intelligence architectures often utilize legacy batch Extract, Transform, Load (ETL) processes that update warehouse data records once a day or in worse cases rely on static, end-of-month database reporting. In this traditional batch model, latency in data processing introduces massive delays between the generation of data and its availability for business analysis. Because delays are caught weeks or days after they occur, the subsequent financial losses and operational bottlenecks cannot be undone. Beyond the technological latency, this reactive model places a massive burden on human analysts to manually hunt for these isolated SLA breaches. Critical delays can then be easily overlooked due to human error from dashboard fatigue. As noted by Gartner, legacy polling systems and batch mechanisms are no longer scalable enough to meet modern real-time operational requirements, proving both too costly and too slow to effectively manage data distribution [3].

Proposed Concept

To resolve this systemic data latency issue, this project introduces the “Automated Vendor SLA Tracker,” an end-to-end, event-driven ETL pipeline and alerting microservice. According to Gartner analysts, Event-Driven Architecture (EDA) currently holds the top position for application integration serving as the foundational element for the next decade of digital transformation [4]. Instead of waiting for monthly reports or daily batch updates, this system’s core architecture continuously ingests unstructured, live JSON data from external sources. To simulate the unpredictable, live data stream of an enterprise delivery vendor, this project utilizes the public Boston MBTA transit JSON API as a proxy. Finally, the architecture closes the operational loop by making this real-time data immediately actionable. The system features an interactive Business Intelligence dashboard using Power BI. This dashboard is connected directly to the database to visualize SLA compliance rates, while an event-driven Python microservice automatically fires formatted webhook alerts to a corporate Slack channel the exact moment a vehicle exceeds its delay threshold.

Related Work

Enterprise supply chain and logistics tracking historically relied on Electronic Data Interchange (EDI) and secure File Transfer Protocol (sFTP) batch feeds. These legacy systems were designed to securely transmit invoices, purchase orders, and daily shipping manifests between corporate entities. Although EDIs are secure and have been standardized in the industry, they are fundamentally constrained by their batch processing nature. According to supply chain integration research, this batch processing completely lacks real-time capabilities, meaning data arrives in scheduled intervals rather than instantly [5]. The architectural limitation means that real-time SLA enforcement is impossible within a traditional EDI framework. Hence, operational managers are only informed of a delayed shipment long after the delivery window has closed and the downstream manufacturing or fulfillment process has already been disrupted.

In recent years, enterprises have attempted to modernize their visibility by overlaying advanced Business Intelligence (BI) platforms, such as Tableau or standard Power BI implementations, onto their data warehouses. While these modern dashboards provide visual analytics and historical tracking of vendor performance, they predominantly operate on a passive monitoring model. That means analysts are still needed to manually refresh datasets and actively hunt for SLA breaches amidst massive tables of compliant data. Industry analysts note that this manual requirement inevitably leads to “dashboard fatigue,” a phenomenon where information becomes so abundant and poorly prioritized that teams mentally check out and important decision making stalls [6]. In a global enterprise environment, relying on a human operator to notice a critical and time sensitive anomaly creates a severe operational vulnerability.

The Automated Vendor SLA Tracker differentiates itself from these existing paradigms by shifting from passive reporting to active, event-driven alerting. While related operational dashboards simply display static metrics, this project implements a true microservice architecture that actively pushes critical intelligence to stakeholders. By combining a live JSON API feed for real time data ingestion with an automated webhook alerting pipeline, this architecture completely removes the human analyst from the detection phase. As outlined by IBM, Event-Driven Architectures (EDAs) allow enterprises to identify system events and respond to them in real time without idling [7]. The system borrows modern DevOps monitoring principles and applies them directly to supply chain logistics, ensuring that SLA breaches are actively reported the exact minute they occur.

System Architecture

The Automated Vendor SLA Tracker is engineered as an end-to-end Extract, Transform, Load (ETL) pipeline coupled with an event-driven alerting microservice. At its core, the architecture is designed to continuously ingest unstructured, high-velocity telemetry data, standardize it into a relational format, and simultaneously serve both a frontend analytic dashboard and a backend alerting daemon. To accurately simulate the unpredictable nature of an enterprise logistics provider, the system utilizes the public Boston MBTA transit JSONAPI as its primary external data source. This API provides a

continuous stream of real time vehicle locations and predicted arrival times. Therefore, it acts as a proxy for real-world enterprise delivery tracking.

The data flow begins with the Ingestion Engine (`api_ingestion.py`), a Python script that queries the external API and extracts the raw, nested JSON payloads. The transformation layer immediately processes this payload, isolating operational variables such as vehicle IDS, target destination, and raw arrival timestamps. A crucial component of this transformation involves datetime mathematics. The system is currently generalized to convert incoming timestamps into the UTC timezone to calculate the exact minutes remaining until arrival. The system then determines if the vehicle has beached a predetermined 5 minute SLA threshold. Once transformed, these standardized performance metrics are loaded into a SQLite relational database (`slaTracker.db`), which acts as the central source for the entire architecture.

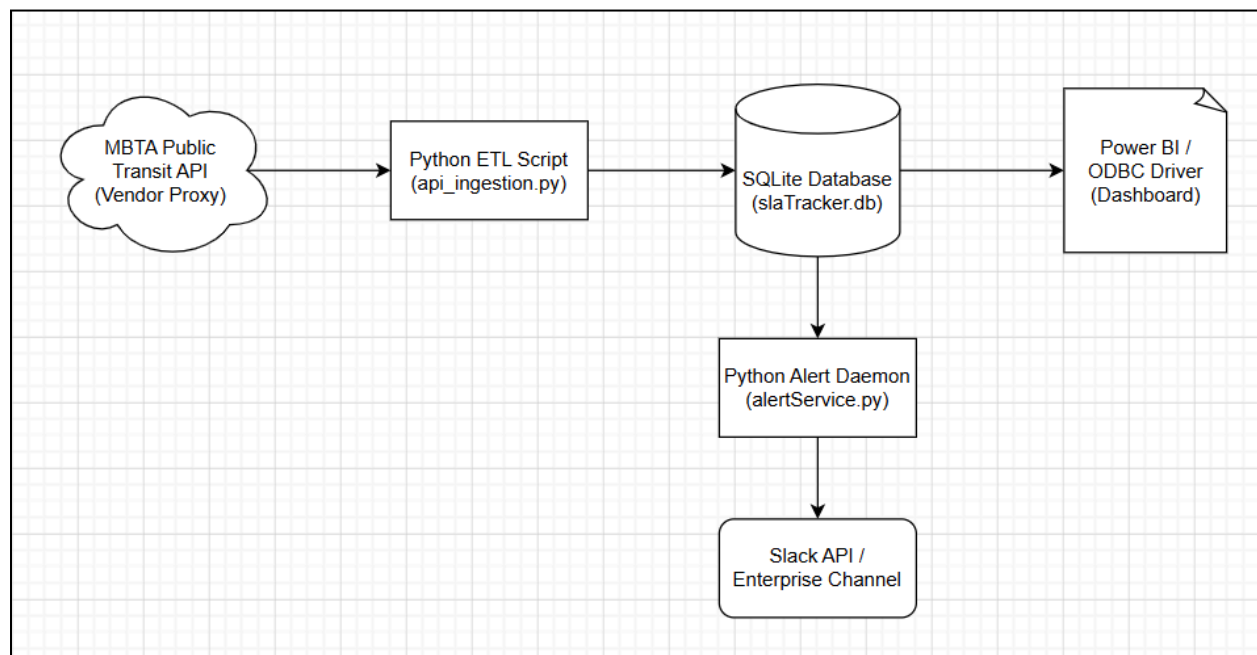


Figure 1: End-to-End System Architecture of the Automated Vendor SLA Tracker

From the SQLite database, the architecture branches into a dual-output layer to serve different business needs. For historical analytics and executive visibility, a localized instance of Power BI connects directly to the database via an Open Database Connectivity (ODBC) driver, rendering the aggregated data into an interactive

dashboard. Simultaneously, the backend Alerting Daemon (alertService.py) continuously polls the database to identify new SLA breaches. When a delay is detected, the microservice utilizes an incoming webhook to push a Markdown alert directly into a corporate Slack channel. A crucial part to this microservice is the inclusion of a state-aware cooldown logic. The microservice records active alerts in the database to throttle duplicate notifications which prevent channel spam when a vehicle remains delayed over consecutive polling cycles.

Enterprise Configuration & Scalability:

In the current local architecture, the 5-minute SLA breach threshold and the 15-minute alert cooldown are statically defined within the Python scripts. However, in a production enterprise environment, these parameters would not be hardcoded. Instead, the system would implement a centralized Configuration Management Database (CMDB) allowing each vendor to have a dynamic profile mapping to their legally negotiated contract. This ensures the alerting logic scaled dynamically to support hundreds of varying vendor contracts without requiring code deployments.

Implementation Methodology

Data Extraction & Transformation Pipeline:

The foundation of the system relies on a Python ingestion engine (api_ingestion.py) engineered to interface with the MBTA public transit JSON API. During the extraction phase, the script utilizes the “requests” library to establish a secure HTTP connection, retrieving live positional data of the trains and the predicted arrival times for each active transit vehicle. Since raw API feeds typically contain missing or malformed data elements, the ingestion script includes exception handling using a Try-Except block to bypass null record without crashing the execution loop.

Once the JSON payload is successfully parsed, the data enters the transformation phase. The most pivotal challenge was standardizing time formats to ensure accurate date-time calculations. In global logistics tracking, standardizing time

formats across different geographic zones is essential to prevent data corruption from daylight savings and regional serval shifts. To achieve system generalizability, the engine intercepts raw timestamps provided by the API and explicitly standardizes all temporal data to Coordinated Universal Time (UTC) using Python's "datetime" module. The script then calculates the delta between the current UTC time and predicted arrival. If this delta exceeds the legally negotiated threshold, a boolean "isBreach" flag is triggered. The standardized data is then formatted into a relational schema and committed to a local SQLite database (slaTracker.db).

Database Configuration & Business Intelligence Integration:

The localized SQLite database serves as the persistent storage layer, safeguarding the historical performance metrics required for long-term vendor evaluation. To make the data visually accessible to executive stakeholders, the architecture utilizes Microsoft Power BI. Because Power BI Desktop can't natively query a localized SQLite file in real time, the implementation required the configuration of an Open Database Connectivity (ODBC) driver.

By establishing a system-level ODBC Data Source Name (DSN), Power BI is able to securely query the SQLite tables via direct SQL statements. Within Power BI, the raw pipeline data is further refined using Data Analysis Expressions (DAX). Custom DAX measures were engineered to dynamically calculate the overall "SLA Compliance Percentage" and visualize the "Average Delay Severity" across different transit routes. This transformed raw ingestion logs into an interactive, enterprise-grade executive dashboard. Additionally, executive and stakeholders can directly extract Excel data sheets of desired metrics for further inspection.

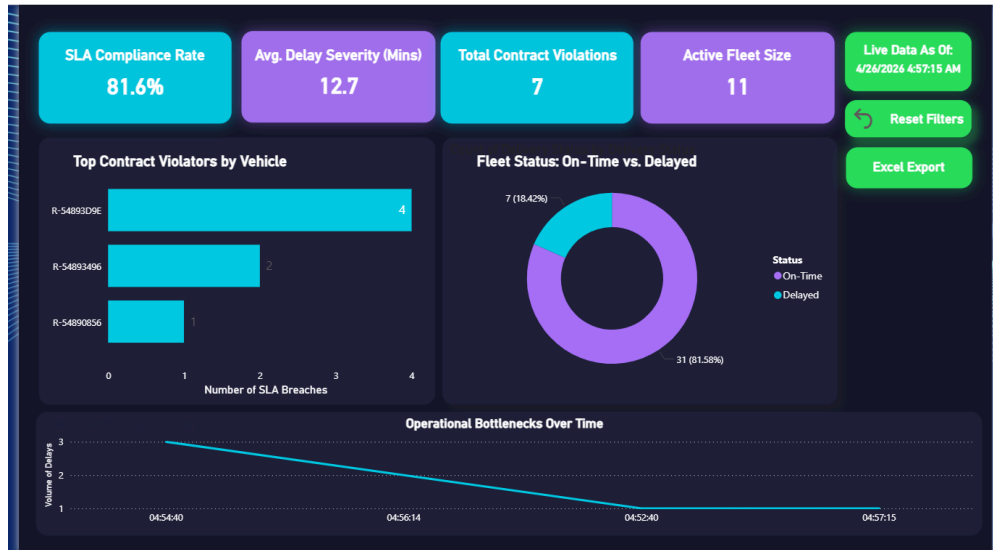


Figure 2: The Power BI dashboard displaying real-time SLA compliance metrics

Alerting Microservice:

While the Power BI dashboard satisfies the need for historical analytics, the architecture's real-time capabilities are driven by an independent Python microservice (alertService.py). This daemon operates concurrently with the ingestion engine, continuously polling the SQLite database for newly flagged SLA breaches. Once the daemon detects a new breach, it constructs a formatted Markdown payload and transmits it via a webhook to a designated enterprise Slack channel.

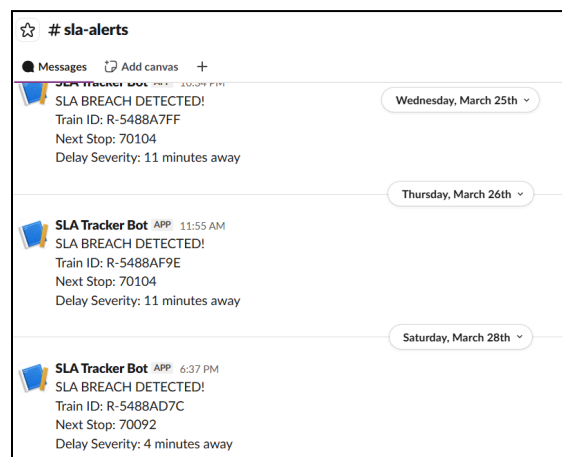


Figure 3: Successful Markdown payload transmitted as a message in a Slack Channel

To prevent dashboard fatigue and spam alerts, this microservice was engineered with state awareness. In a naive implementation, a vehicle delayed by 13 minutes would trigger a duplicate Slack alert every minute the pipeline runs. To solve this, the microservice implements an “alertState” tracking table. When an alert is successfully fired, the vehicle ID and timestamp of the sent alert is logged in the table. Before sending any future alerts, the daemon checks this state table. If the vehicle had been alerted within the past 15 minutes, the script throttles the webhook, ensuring stakeholders are notified of genuine anomalies without continuously being bombarded by duplicate data.

Evaluation and Results

System Load Testing:

Because the architecture utilizes a lightweight local SQLite database to juggle a frontend BI tool and a backend daemon simultaneously, it is necessary to evaluate the system’s concurrency limits. To simulate a massive spike in data, a stress testing script (streeTest.py) was developed. The testing protocol inserts 1,000 artificial SLA breach records into the database in a single execution burst. Even with the background alert daemon concurrently active during this burst, the overload test was able to process, commit, and alert all 1,000 database inserts in 0.03 seconds. This proved that the data engineering was highly optimized and eliminates the risk of database functionality failing during peak operational traffic.

Alert Throttling Validation:

To validate the alerting logic, the system was subjected to a simulated “alert spam” scenario. The testing involved executing the live api_ingestion.py script back to back within a one minute window, forcing the system to re-ingest data for vehicles that were already actively delayed. During the initial ingestion run, the Slack webhook successfully executed. The second run then correctly identified the active state of delayed vehicles as the webhook was successfully suppressed having zero duplicate

Slack notifications sent out. After the 15 minute cooldown, an additional message was sent proving the logic operated perfectly under live conditions.

Future Work

The short term future work must focus on unifying the execution pipeline to achieve true continuous monitoring. Currently, the pipeline is fragmented as the `api_ingestion.py` script operates manually pulling a single snapshot of transit data each time it is run. Hence to make the system fully autonomous, the ingestion engine and alerting microservice must be refactored to run continuously in the background. This can be reached by creating an orchestrator script that schedules sleep intervals for how often the business wants to check for SLA Compliance Violations and sets up the SQLite schema and dependencies for new users. Then testing must be done to prevent API timeout or networks from interrupting the continuous runtime.

Looking toward long term enterprise scalability, the architecture would greatly benefit from cloud migration and advanced analytical integrations. Transitioning from a local SQLite database to a cloud warehouse would allow the system to handle resources better during higher traffic periods. This would also resolve a current limitation that comes with the manual dashboard refreshes. By deploying the reporting file to the enterprise Power BI Service and connecting it to the cloud data warehouse over the local database, the dashboard would update autonomously eliminating the need for human intervention. Additionally, our historical tracking data can be used to train predictive machine learning models to preemptively mitigate loss by forecasting delays before SLA breaches occur.

Project Deliverables

Github Repository: <https://github.com/ryandude99/Automated-Vendor-SLA-Tracker>

- The Github Repository includes all Python, Power BI, and SQLite Database coding files
 - databaseSetup.py: Sets up SQLite Database
 - api_ingestion.py: Python ETL Script
 - alertService.py: Python Alert Daemon
 - SLA_Tracker_Dashboard.pbix: Power BI Dashboard
 - stressTest.py: Database Concurrency Validation
 - INSTRUCTIONS.md: User Instructions to Run Application
- Presentations Materials are included in the GitHub Repository under the name *Automated_Vendor_SLA_Tracker_Presentation.pptx*

Presentation Video: <https://youtu.be/-ctgOkICiaU>

Skill Learning

Throughout the lifecycle of this project, several meta-skills were developed to navigate ambiguity, manage technical constraints, and design for human users. The following meta-skills were learned:

- **Architectural Pivoting & System Resilience:** One of the crucial skills developed was the ability to architecturally pivot my design in response to external technical failures. During the initial planning phase, the architecture was designed to ingest data from the MARTA transit API. However, it had severe connectivity issues and unreliable JSON payloads to accurately simulate a global logistics feed. Rather than settling to a lower standard and drudging through the difficulties that came with using this API, I decided to pivot to the Boston MBTA JSON API. This required rapidly reading new technical documentation, restructuring the ingestion logic, and adapting the data models. This experience reinforced that external dependencies will fail and engineering resilience comes from well designed, generalized systems that can swap data sources without fully compromising the core ETL pipeline.
- **Managing Enterprise Technical Constraints:** Building a local prototype of an enterprise system proved to have hidden complexities. A major learning curve occurred while using Business Intelligence as Power BI Desktop could not hold my assumption that it could continuously query a local SQLite database file. After in-depth research, I discovered it was structurally impossible to implement this functionality. Instead I had to configure a system level Open Database Connectivity (ODBC) driver to bridge the Python database to the Microsoft frontend dashboard. This helped me understand that modern system communications require more than just writing well made code, but instead requires knowledge on how to connect systems together.
- **Human Centric Design:** During the early iterations of the alerting microservice, although alerts were successfully outputted, having continuous duplicate outputs proved to actively contribute to dashboard fatigue. Understanding that the end user would not benefit from these constant messages required a redesign of the daemon to have cooldown timers to limit duplicate notifications. Additionally, it

was important to design a dashboard that fit the needs of the analysts, businesses, and stakeholders using them. It proved important not to clutter the dashboard with irrelevant metrics and charts and ensure the implementation of important functionalities that would better the user experience. This challenge fundamentally shifted the project's perspective towards a human centric design to respect the user's attention and cognitive load.

References

- [1] <https://www.supplychainbrain.com/blogs/1-think-tank/post/40527-how-to-deploy-multi-tier-supply-chain-intelligence-for-resilience>
- [2] <https://onspring.com/resources/blog/tier-2-tier-3-supply-chain-risk-visibility/>
- [3] <https://www.diffusiondata.com/gartner-on-event-driven-architecture/>
- [4] <https://solace.com/blog/gartner-on-event-driven-architecture/>
- [5] <https://www.edgeverve.com/tradeedge/blogs/edi-api-supply-chain-integration-strategy/>
- [6] <https://www.sigmacomputing.com/blog/data-fatigue>
- [7] <https://www.ibm.com/think/topics/event-driven-architecture>